

1) 주어진 공개 파라미터와 출력값, 로그 레코드(telemetry.csv)를 이용하여 다음 출력값 r_3 를 예측하시오.

주어진 로그 레코드(telemetry.csv)를 확인하면 round, scale, offset, summary가 있다. 이를 문제 조건에 따라 해석하면 다음과 같은 관계식을 추론할 수 있다.

$$summary = \lfloor \frac{((scale \cdot d) + offset) \bmod n}{2^{20}} \rfloor$$

관계식을 바탕으로 로그 레코드에 있는 두개의 행을 이용해 다음과 같은 연립방정식을 구성할 수 있다. 이때 e 는 제거된 20비트를 보정하는 0이상 2^{20} 미만의 오차항이다.

$$summary_0 \cdot 2^{20} + e_0 = (scale_0 \cdot d + offset_0) \bmod n \dots ①$$

$$summary_1 \cdot 2^{20} + e_1 = (scale_1 \cdot d + offset_1) \bmod n \dots ②$$

식 ①을 d 에 대해 정리한 후 식 ②에 대입하여 정리하면 다음과 같다.

$$A = (scale_1 \cdot scale_0^{-1}) \bmod n$$

$$B = [A \cdot (summary_0 \cdot 2^{20} - offset_0) - (summary_1 \cdot 2^{20} - offset_1)] \bmod n$$

$$e_1 = (A \cdot e_0 + B) \bmod n$$

0, 1 round 행의 값을 사용하여 조건 범위 $0 \leq e < 2^{20}$ 를 만족하는 순서쌍 (e_0, e_1) 을 찾으면 단 1개의 쌍을 찾을 수 있다. 해당 쌍을 사용해 d 를 찾을 수 있고 해당 값으로 2~5 round 행 검증, $P = dQ$ 검증을 하면 모두 통과한다. 해당 과정의 의사코드와 d 값은 다음과 같다.

```
S_i ← summary_i × 2^20
inv_s0 ← scale_0^{-1} mod n
A ← scale_1 × inv_s0 mod n
C ← S_1 - A × (S_0 - offset_0) - offset_1 mod n

for e0 in range(2^20):
    e1 ← (A × e0 - C) mod n
    if 0 ≤ e1 < 2^20:
        d_cand ← inv_s0 × (S_0 + e0 - offset_0) mod n
        if ∀i∈{2..5}: [(scale_i × d_cand + offset_i) mod n / 2^20] = summary_i:
            return d_cand
```

코드 1. d 를 찾는 의사코드(solve.py)

$d = 0x1c3cdd6b221806db0a7b28$

한편 $P = d \cdot Q$ 를 활용하여 다음과 같이 전개할 수 있다.

$$R_i = s_{i+1}Q \text{라 하자}$$

$$R_i = s_{i+1}Q \Leftrightarrow dR_i = ds_{i+1}Q \Leftrightarrow dR_i = s_{i+1}P$$

$$s_{i+2} = X(s_{i+1}P) = X(dR_i)$$

$$R_{i+1} = s_{i+2}Q = X(dR_i)Q$$

수식을 해석하자면 i 번째 난수를 만드는데 사용한 점 R_i 를 알고 백도어 d 를 알면 R_{i+1} 을 계산하여 다음 $i+1$ 번째 난수를 구할 수 있다.

r_0 값으로 R_0 의 x좌표 상위 72비트를 확정하고, 하위 16비트는 브루트포스하여 후보군을 생성한다. 이렇게 얻은 후보들로 r_1 을 계산한 뒤 문제에서 주어진 실제 값과 비교하여 최종 내부 상태를 결정 할 수 있다. 다음은 의사 코드와 찾은 r_3 값이다. 총 65536개의 후보가 있으며 21304번째 후보에서 r_3 을 찾을 수 있었다. 소요시간은 1분 미만이었다.

```

for low in range(2^16):
    x ← (r0 << 16) | low
    if x ≥ p: continue
    rhs ← x^3 + axx + b mod p
    if pow(rhs, (p-1)/2, p) ≠ 1: continue
    y ← pow(rhs, (p+1)/4, p)
    for y_cand in {y, p-y}:
        R0 ← (x, y_cand)
        s2 ← X(d·R0)
        R1 ← s2·Q
        if X(R1) >> 16 ≠ r1: continue
        s3 ← X(d·R1)
        R2 ← s3·Q
        s4 ← X(d·R2)
        R3 ← s4·Q
    return X(R3) >> 16

```

코드 2. r_3 를 찾는 의사코드(predict.py)

$r_3 = 0x2443c8daf1a9d52b09$